Time Complexity

Sample Problems

Determine the **Big-O** complexity of the following code snippets.

Easy Level

Q1:

```c
for (int i = 0; i < n; i++) {
    printf("Hello, World!\n");
}
```

Q2:

```c
void printFirstElement(int arr[], int n) {
    printf("%d\n", arr[0]);
}
```

Q3:

```c
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("%d, %d\n", i, j);
    }
}
```

Medium Level

Q4:

```c
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        for (int k = 0; k < 10; k++) {
            printf("%d, %d, %d\n", i, j, k);
        }
    }
}
```

Q5:

```
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}
```

Q6:

```
void printElements(int arr[], int n) {
    for (int i = 1; i < n; i *= 2) {
        printf("%d\n", arr[i]);
    }
}
```

Hard Level

Q7:

```
void foo(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            printf("%d, %d\n", i, j);
        }
    }
}
```

Q8:

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

Q9:

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        printf("%d, %d\n", i, j);
    }
}
```

Q10:

```
void combinedLoop(int n) {
      for (int i = 0; i < n; i++) {
      for (int j = 1; j < n; j *= 2) {
            printf("%d, %d\n", i, j);
      }
   }
}
```

Q11:

```
void conditionalLoop(int n) {
      for (int i = 0; i < n; i++) {
         for (int j = 0; j < n; j++) {
            if (i == j) {
                  break;
            }
            printf("%d, %d\n", i, j);
      }
   }
}
```

Q12:

```
void func(int n) {
      if (n <= 1) return;
      func(n / 2);
      func(n / 2);
      func(n / 2);
}
```

Q13:

```
int recursiveFunction(int n) {
      if (n <= 1) return 1;
      return recursiveFunction(n / 2) + recursiveFunction(n / 2) + n;
}
```

Q14:

```
void complexNestedLoops(int n) {
      for (int i = 0; i < n; i++) {
      for (int j = 0; j < i; j++) {
            for (int k = 0; k < j; k++) {
                  if (k % 2 == 0) {
                  printf("%d, %d, %d\n", i, j, k);
                  }
            }
      }
      }
}
```

Q15:

```
int factorial(int n) {
      if (n <= 1) return 1;
      return f(ceiling(sqrt(n)));
}
```

Solution-Q15:

To solve this recurrence, observe how n decreases in each step:

- In the first step, the input reduces to $\text{ceiling}(\sqrt{n})$.
- In the second step, the input becomes $\text{ceiling}(\sqrt{\sqrt{n}})$.
- After k steps, the input will be approximately $n^{1/2^k}$.

The recursion continues until $n^{1/2^k} \leq 1$, i.e., until k is such that:

$$n^{1/2^k} \leq 1 \Rightarrow 1/2^k \log n \leq 0 \Rightarrow 2^k \geq \log n$$

Taking the logarithm of both sides:

$$k \geq \log \log n$$

Some Tips:

1. **Identify the most time-consuming operation** in the code and focus on how it scales with the input size.
2. **Watch for nested loops** and recursions—they typically indicate polynomial or exponential complexity.
3. **Consider constant factors** (like loops that run a fixed number of times) and ignore them in Big-O notation.
4. **Practice decomposing complex loops** by analyzing the inner and outer loops separately.
